

EDS Lab Assignments

Practical1

Name: Harshal Dhanait

PRN: 202501040049

1. Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m * v$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input())
velocity = float(input())
momentum = mass * velocity
print(f"{momentum:.2f} kgm/s")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '1.1.1. Calculate Momentum' is visible, including the formula $p = m \times v$ and input/output specifications. The main editor shows the Python code for calculating momentum. The right sidebar displays test results, indicating that 2 out of 2 shown test cases passed, with an average time of 0.007s and a maximum time of 0.009s. The test cases show expected and actual outputs of 50.00 kgm/s.

```
1 m = float(input())
2 v = float(input())
3 momentum = m*v
4 print(f"{momentum:.2f}kgm/s")
```

Test Results:

Test Case	Expected Output	Actual Output	Status
Test case 1	50.00	50.00	Passed
Test case 2	50.00	50.00	Passed

2. Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid"

Code:

```
import math
n = int(input())
sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
    print(f"{sq}")
elif n>=10 and n<=99:
    print(f"{sq_root:.2f}")
elif n>=100 and n<=999:
    print(f"{cb_root:.2f}")
else:
    print("Invalid")
```

Output:

The screenshot displays the CODETANTRA IDE interface. On the left, the problem statement is visible, including constraints ($1 \leq n \leq 999$), input/output formats, and sample test cases. The main editor shows the Python code for conditional calculation based on the number of digits. The code uses `math.sqrt()` for two-digit numbers and `math.cbrt()` for three-digit numbers, both rounded to two decimal places. The output section shows the execution results: 4 out of 4 shown test cases passed, and 3 out of 3 hidden test cases passed. The test cases are listed with their expected and actual outputs, all of which are correct.

```
1 n = int(input())
2 if n < 10:
3     print(n*n)
4 elif (n > 9) and (n < 100):
5     print(f"{n**(0.5):.2f}")
6 elif (n > 99) and (n < 1000):
7     print(f"{n**(1/3):.2f}")
8 else:
9     print("Invalid")
```

Execution Results:

- Average time: 0.003 s, Maximum time: 0.008 s
- 3.29 ms, 5.00 ms
- 4 out of 4 shown test case(s) passed
- 3 out of 3 hidden test case(s) passed

Test Case 1: Expected output: 81, Actual output: 81

Test Case 2: Expected output: 3.00, Actual output: 3.00

Test Case 3: Expected output: 10.00, Actual output: 10.00

3. Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement and input/output formats are shown. The main editor area contains the Python code for calculating the number of days between two dates. The code uses the datetime module to parse the input dates and calculate the difference. The output shows the number of days between the two dates, which is 123.

```
1 from datetime import date
2
3 from datetime import datetime
4
5 date1 = input().strip()
6 date2 = input().strip()
7
8 d1 = datetime.strptime(date1, "%Y-%m-%d")
9 d2 = datetime.strptime(date2, "%Y-%m-%d")
10
11 difference = d2-d1
12 print(difference.days)
13
14
```

Test case results:

Test Case	Expected Output	Actual Output
Test case 1	123	123
Test case 2	123	123

4. Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input())
reverse=int(str(n)[::-1])
print(reverse)
```

Output:

The screenshot displays a coding environment with the following components:

- Header:** CDETANTRA logo, navigation links (Home, Learn Anywhere), user profile (202501040049@mitaoe.ac.in), and a Logout button.
- Problem Statement:** 1.1.4. Reverse a Number. The instruction is to write a Python program to reverse the digits of the number and print the reversed number.
- Input Format:** The input is a single integer.
- Output Format:** The output prints a single integer, which is the reversed number.
- Code Editor:** Contains the following Python code:

```
1 n = int(input())
2 reverse = int(str(n)[::-1])
3 print(reverse)
```
- Test Results:**
 - Average time: 0.004 s, Maximum time: 0.006 s.
 - 3.80 ms, 6.00 ms.
 - 2 out of 2 shown test case(s) passed.
 - 3 out of 3 hidden test case(s) passed.
 - Test case 1: Expected output 5357, Actual output 5357.
 - Test case 2: Expected output 7635, Actual output 7635.
- Footer:** Navigation buttons: < PREV, RESET, SUBMIT, NEXT >.

5. Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

Code:

```
n = int(input())  
for i in range(1,11):  
    print(n, "x", i, "=", n*i)
```

Output:

The screenshot displays a coding environment with the following components:

- Header:** CDE TANTRA logo, navigation links (Home, Learn Anywhere), user profile (202501040049@mitaoe.ac.in), and a Logout button.
- Problem Statement (1.1.5. Multiplication Table):**
 - Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.
 - Input Format:** The first line of input contains an integer that represents the number for which the multiplication table is to be printed.
 - Output Format:** Print the multiplication table for the given number in the format: `<number> x <multiplier> = <result>`
 - Note:** The character "x" is used in the output to represent multiplication. Refer to the visible test-cases for more clarity.
 - Sample Test Cases:** A button to view sample test cases.
- Code Editor:** Contains the following Python code:

```
n=int(input())  
for i in range(1,11):  
    print(n,"x",i,"=",n*i)
```
- Test Results:**
 - Average time: 0.006 s, Maximum time: 0.012 s.
 - 2 out of 2 shown test case(s) passed.
 - 2 out of 2 hidden test case(s) passed.
- Test Case Details:**

Test case 1	Expected output	Actual output
1	8 x 1 = 8	8 x 1 = 8
2	8 x 2 = 16	8 x 2 = 16
3	8 x 3 = 24	8 x 3 = 24
4	8 x 4 = 32	8 x 4 = 32
5	8 x 5 = 40	8 x 5 = 40
- Footer:** Navigation buttons: < PREV, RESET, SUBMIT, NEXT >.

1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
    if any(mark<40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)

num_courses = int(input())
marks = list(map(int,input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
    aggregate_percentage, grade = result
    print("Aggregate Percentage:",f"{aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```

Output:

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases

passorFa...

```
1 # write your code here
2 def cal(marks, total_courses):
3     if any(mark < 40 for mark in marks):
4         return "Fail"
5     total_marks = sum(marks)
6     aggregate_percentage = (total_marks / (total_courses * 100)) * 100
7     if aggregate_percentage > 75:
8         grade = "Distinction"
9     elif aggregate_percentage >= 60:
10        grade = "First Division"
11    elif aggregate_percentage >= 50:
12        grade = "Second Division"
13    elif aggregate_percentage >= 40:
14        grade = "Third Division"
```

Average time: 0.006 s, Maximum time: 0.009 s, 5 out of 5 shown test case(s) passed, 5 out of 5 hidden test case(s) passed

Test case 1: Expected output: 55.65 78.89, Actual output: 55.65 78.89, Aggregate Percentage: 71.75, Grade: First Division

Test case 2: Expected output: 55.65 78.89, Actual output: 55.65 78.89, Aggregate Percentage: 71.75, Grade: First Division

1.2.2 Fibonacci Series

Problem Statement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):
    if n==0:
        return a
    else:
        return fibonacci(n-1,b,a+b)
```

```
n=int(input())
for i in range(n):
    print(fibonacci(i),end=" ")
```

Output:

The screenshot shows the CODETANTRA online IDE interface. On the left, the problem statement for '1.2.2. Fibonacci Series' is displayed, requiring a Python program to print the first n terms of the Fibonacci series using recursion. The input format is a single integer n , and the output format is a single line of the first n terms separated by spaces. Below the problem statement is a 'Sample Test Cases' section.

The main editor shows a Python file named 'fibonacci...' with the following code:

```
1 # def fibonacci(n):
2 #     # write the code..
3 #     return fibonacci(...)
4 # n = int(input())
5 # for i in range(1, n+1):
6 #     print(fibonacci(i), end=" ")
7 def fib(n, a=0, b=1):
8     if n==0:
9         return a
10    else:
11        return fib(n-1, b, a+b)
12 n = int(input())
13 for i in range(n):
14    print(fib(i), end=" ")
```

At the bottom right, the execution results are shown: '2 out of 2 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. Below this, the details for 'Test case 1' are visible, showing the expected output '5' and the actual output '0 1 1 2 3'.

1.2.3 Pattern-1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("*", end=" ")
    print()
```


Output:

The screenshot shows the CODETANTRA online IDE interface. On the left, the problem statement for '1.2.3. Pattern - 1' is displayed, asking for a Python program to print a right-angled triangle of asterisks. The input format specifies an integer for the number of rows. The output format shows a sample pattern. The main editor on the right contains the following Python code:

```
1 n=int(input())
2 for i in range(1,n+1):
3     print('*'*i)
```

Below the code editor, the execution results are shown: '2 out of 2 shown test case(s) passed' and '4 out of 4 hidden test case(s) passed'. The average time is 0.005 s and the maximum time is 0.008 s. A 'Test case 1' section shows the expected and actual output, both being a right-angled triangle of asterisks.

1.2.4 Pattern-2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Output:

CODETANTRA

Home Learn Anywhere

202501040049@mitaoe.ac.in Support Logout

1.2.4. Pattern - 2

02:01

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

numberP...

```
1 n=int(input())
2 for i in range(1,n+1):
3     a=1
4     for j in range(1,i+1):
5         print(a,end=" ")
6         a+=1
7     print("")
```

Average time0.004 s3.75 msMaximum time0.005 s5.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 13 msDEBUG

Expected output	Actual output
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

TerminalTest cases

PREV

RESET

SUBMIT

NEXT